Jessica AS-2 13/11/26

Do not write in this margin

Q1(a)
9
15
5
11
6

(b)

i	a[i] before	total after	a[i] after
0	2	2	2
1	6	8	8
2	5	13	13
3	8	21	21

Output: 8 21

c) The index j is out of the boundary. It's a syntax error.

d) It only changes the value of the parameter v. But it doesn't change the value in the array. It is pass-by-value.

Q2)

```
FUNCTION(nums: ARRAY[1:n] OF INTEGER) RETURNS INTEGER
    DECLARE cnt, index :INTEGER
    cnt ← 0
    index ← 1
    FOR index ← 1 TO n
        IF nums[index] MOD 2 = 0 THEN
            cnt ← cnt+1
        ENDIF
    NEXT index
    RETURN cnt
ENDFUNCTION
```

write this margi

```
DECLARE str : STRING
```

b)

```
DECLARE temp[1:8] : ARRAY OF INTEGER
DECLARE index, ~~cnt~~ cnt, ~~aver~~ : INTEGER
DECLARE aver : REAL
FOR index ← 1 TO 8
    INPUT temp[index]
NEXT index
aver ← average(temp~~[1:8]~~)
OUTPUT aver
~~OUTPUT temp[1]~~
cnt ← 1
~~cnt ← 1~~
~~aver ← 1~~
FOR cnt ← 1 TO 8
    IF temp[cnt] > aver THEN
        str ← str + NUM_TO_STR(temp[cnt]) + " "
    ENDIF
NEXT cnt
OUTPUT str
```

Q3

PB

a)

```
[6, 5, 14, 9, 11, 10]
6 9 11
```

b) It makes the procedure ~~influe~~ influence the array A ~~but not~~ instead of the parameter Data. It sends the position of the value of A.

If removed, the array A in the main program will not change, the change is only in the procedure.

c)

```
PROCEDURE Replace(BYREF Data : ARRAY[1:8] OF INTEGER,
                  OldVal : INTEGER, NewVal : INTEGER)
    DECLARE index : INTEGER
    FOR index ← 1 TO 8
```

this
margi

```
        IF Data[index] = OldVal THEN
            Data[index] ← NewVal
        ENDIF
    NEXT index
ENDPROCEDURE
```

d)
```
PROCEDURE Swap(BYREF X : INTEGER, BYREF Y : INTEGER)
    DECLARE exc : INTEGER
    exc ← X
    X ← Y
    Y ← exc
ENDPROCEDURE

DECLARE A, B : INTEGER
~~CALL Swap~~
A ← 5
B ← 9
CALL Swap(A, B)
OUTPUT A, " ", B

//The output is 9 5
```